Quash Command Shell Implementation Report

Student Names: Brianna Stewart

Course: Operating Systems

Task 1: Prompt Formatting and Built-In Commands

The initial phase of the project focuses on establishing the core read-eval-print loop of the shell. A professional command line interface must maintain contextual awareness of the file system by displaying the current working directory in the prompt. To achieve this, Quash uses the getcwd() system call during every iteration of the main loop to retrieve the absolute path of the current directory. This path is then formatted into the required prompt structure consisting of the path followed by a greater than sign.

User input is captured from standard input using the fgets() function to prevent buffer overflow vulnerabilities. Once a command line is read, a custom tokenization function processes the string using strtok() to separate it into an argument vector. To ensure that environment variables can be utilized seamlessly across all commands, the tokenizer checks each token for a leading dollar sign. When a token starts with this character, the shell uses the getenv() library function to look up the corresponding environment variable value and substitutes it into the argument array.

Quash handles several core directives natively within the parent process rather than forking external binaries, because these commands alter the state of the shell execution environment itself. These built-in commands include:

- **cd**: Changes the current working directory using the chdir() system call, defaulting to the user home directory if no path is provided.
- **pwd**: Prints the current working directory path retrieved via getcwd().
- **echo**: Prints text messages and environment variable values by iterating over the argument tokens.
- **env**: Prints all active environment variable pairs by accessing the global environ array, continuing until it hits a terminating null string.
- **setenv**: Sets or updates environment variables using the setenv() function with overwrite enabled.
- **exit**: Terminates the shell program cleanly.

Task 2: Adding Process Management and External Execution

For any command entered by the user that is not a built-in instruction, Quash must delegate execution to an external program file stored on the file system. This requires the use of process forking and program loading. When a non-built-in command is encountered, the shell invokes the fork() system call to duplicate the calling process into a parent and a child.

Inside the child process, Quash calls execvp() to load and execute the specified program file, passing along the argument vector. The execvp() function searches the system path directories for the executable binary and replaces the child process memory image with the new program. If the file does not exist or

execution fails, execvp() returns a negative value, at which point the shell uses perror() to output an appropriate error message before the child process exits. Meanwhile, the parent process acts as a supervisor, using the waitpid() system call to block execution until the child process terminates, ensuring proper synchronization.

Task 3: Background Processes

Standard shells require the ability to run long tasks asynchronously so that the user does not lose access to the command prompt while a program executes. Quash implements background process execution by inspecting the end of the command line for an ampersand symbol.

When the tokenizer detects an ampersand at the end of the input, it strips the symbol from the argument array and sets an internal background flag. When forking the process, the parent shell checks this flag. If the task is designated for the background, the parent process prints the assigned process ID and returns immediately to print the next prompt, bypassing the blocking waitpid() call.

Task 4: Signal Handling

A significant limitation of basic shell implementations is that sending a keyboard interrupt via Control-C causes the operating system to terminate both the running foreground child process and the shell itself. To prevent the shell from quitting unexpectedly when a user interrupts a program, Quash implements a dedicated signal handler for the SIGINT signal.

By utilizing the signal() system call, the shell intercepts SIGINT events. When the user presses Control-C, the signal handler checks whether a foreground process is currently active. If a foreground child is running, the signal is forwarded exclusively to that child process, keeping the shell session secure and immediately returning control to the command prompt.

Task 5: Process Timeout Enforcement

To stop foreground processes from running forever or hanging the shell, we implemented a ten-second timeout. Before waiting on a foreground process, we set an alarm for ten seconds. If the process finishes in time, we cancel the alarm. If it takes longer than ten seconds, a SIGALRM signal triggers a handler that uses kill() to terminate the process.

Task 6: Input and Output Redirection

Quash extends its utility by supporting standard output redirection through the use of the greater-than symbol. When a user specifies redirection on the command line, the shell parses the operator and identifies the destination filename.

Inside the child process, prior to calling execvp(), Quash opens the specified output file with appropriate creation and truncation flags. It then utilizes the dup2() system call to redirect standard output to the newly opened file descriptor. Consequently, any text output generated by the executed command is written directly into the target file rather than appearing on the terminal screen.

Conclusion

Working on Quash was a great way to learn about systems programming in C. By implementing process forking, execution, signals, timers, and redirection, we gained a solid understanding of how operating systems manage tasks and user commands.